

결과물

1. 프로젝트 개요 및 환경

- **실행 환경:** Ubuntu 24.04 LTS (VMware 기반 가상화 환경 환경 구성).
- **프로젝트 목표:** KVM, MicroCeph, Linux Network Namespace를 활용한 IaaS 축소판 인프라 자동 구축 .

2. 아키텍처 및 구현 상세

🗄️ Storage Layer	🌐 Network Layer	🖥️ Compute Layer
• MicroCeph 단일 노드 클러스터 구축 • Pool: vm_pool (replicated) • Ubuntu Cloud Image → raw 변환 → RBD import • 각 VM 전용 RBD 이미지 (vm1-disk, vm2-disk, vm3-disk) • <code>rbd map</code> → <code>/dev/rbd*</code> 커널 블록 디바이스로 노출 • 🚫 로컬 <code>qcow2</code> 파일 부팅 완전 금지	• vnet-A: Linux Bridge (br-vnet-a) — vm-1, vm-2 연결 • vnet-B: Linux Bridge (br-vnet-b) — vm-3 연결 • 동일 CIDR 192.168.0.0/24 사용, L2 세그먼트 분리 • 방화벽 규칙이 아닌 브리지 물리 격리로 구현 • vm-1 ↔ vm-2 통신 가능 / vm-3 격리 확인 • <code>iproute2</code> + <code>iptables</code> 만 사용 (No NetworkManager)	• QEMU + KVM 하드웨어 가속 가상화 • Cloud-init seed.iso 동적 생성 → 정적 IP 주입 • vm-1: 192.168.0.1 vm-2: 192.168.0.2 vm-3: 192.168.0.3 • <code>-daemonize</code> 백그라운드 실행 • VNC 포트 제공: 5901(vm1) 5902(vm2) 5903(vm3) • <code>virsh console</code> 또는 VNC 클라이언트로 접근 가능

00_prereq → 01_microceph → 02_network → 03_vm → 04_test → cleanup

- **Storage (MicroCeph):**
 - 과제 조건에 따라 로컬 파일 부팅을 피하기 위해, Ubuntu Cloud Image를 Raw 포맷으로 변환 후 Ceph Pool(`vm_pool`)에 RBD로 Import함 .
 - QEMU가 RBD를 인식할 수 있도록 `rbd map` 명령어를 사용해 커널 블록 디바이스 (`/dev/rbd*`) 형태로 마운트하여 부팅 디스크로 제공함.
- **Network (VNet & L2 격리):**
 - `vnet-A` 와 `vnet-B` 를 각각 독립된 Linux Bridge 인터페이스로 생성함 .
 - 방화벽(`iptables`) 규칙에 의한 차단이 아닌, 물리적인 L2 스위치를 분리하는 방식과 리눅스 네트워크 네임스페이스를 활용해 근본적인 네트워크 격리를 구현함 .

- **Compute (KVM & QEMU):**

- Cloud-init(초기화 데이터 `seed.iso`)을 스크립트 내에서 동적으로 생성해, VM 부팅 시 `192.168.0.x/24` 대역의 정적 IP와 SSH 계정 정보를 자동 주입함 .
- `qemu-system-x86_64` 를 `-daemonize` 옵션으로 백그라운드 실행하고, `-vnc` 옵션을 추가해 포트(5901~5903)로 콘솔 접근을 제공함.

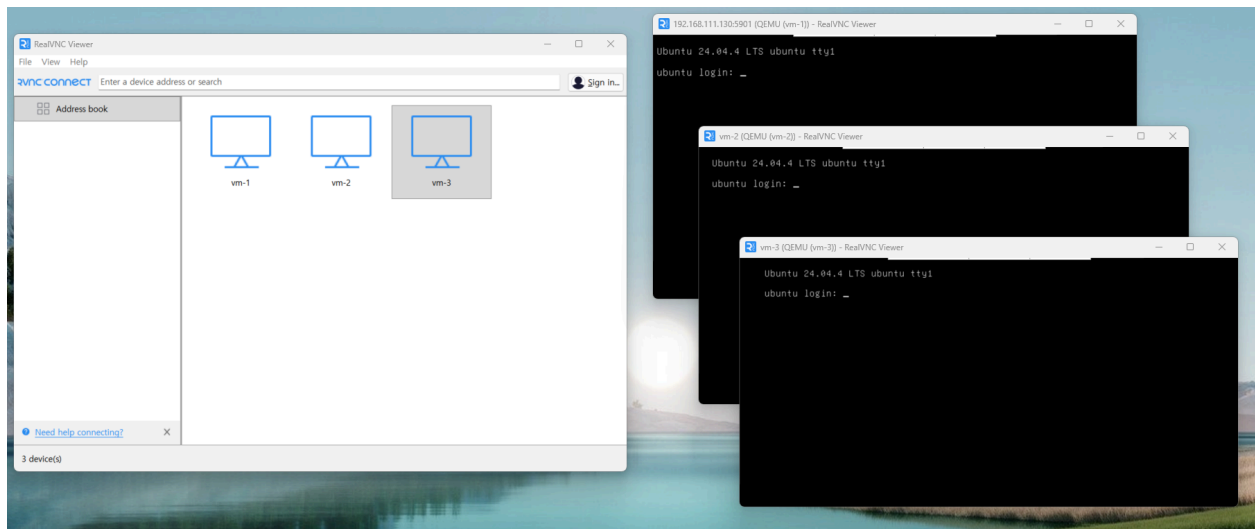
3. 스크립트 실행 흐름 및 방법

본 프로젝트는 완전한 자동화를 위해 5단계의 실행 스크립트와 1개의 자원 정리 스크립트로 모듈화하여 구성했습니다. 아래 순서대로 스크립트를 실행하여 전체 인프라를 프로비저닝하고 검증합니다.

- **00_prereq.sh (사전 환경 구성):** QEMU, MicroCeph, iproute2 등 인프라 구축에 필요한 필수 패키지들을 호스트(Ubuntu 24.04)에 자동 설치합니다.
- **01_microceph.sh (스토리지 프로비저닝):** MicroCeph 단일 노드 클러스터를 초기화하고, Loop device 기반의 가상 OSD를 추가합니다. 이후 Ubuntu Cloud Image를 다운로드 하여 Raw 포맷으로 변환한 뒤, 각 VM이 부팅할 개별 RBD 이미지로 Import 합니다.
- **02_network.sh (네트워크 격리 구성):** `iproute2` 를 사용하여 호스트 내부에 두 개의 독립된 물리적 L2 스위치 역할을 하는 Linux Bridge(`vnet-A`, `vnet-B`)를 생성하고 QEMU 접근 권한을 부여합니다.
- **03_vm.sh (컴퓨팅 자원 실행):** Cloud-init(`seed.iso`) 설정 파일을 동적으로 생성하여 각 VM에 요구사항에 맞는 정적 IP(192.168.0.x/24)와 SSH 계정을 주입합니다. 이후 생성된 RBD 이미지를 커널 블록 디바이스로 매핑하여 QEMU/KVM 기반의 가상머신 3대를 백그라운드(`daemonize`)로 실행합니다.
- **04_test.sh (자동화 검증):** Linux Network Namespace(netns)를 활용해 가상의 테스트 노드를 생성하고, 요구사항에 명시된 통신 규칙(vm1-vm2 통신 가능, vm3 격리)을 Ping 테스트로 자동 검증합니다.
- **cleanup.sh (자원 라이프사이클 관리):** 실행 중인 VM 강제 종료, RBD 매핑 해제 및 이미지 삭제, 브릿지 삭제 등을 수행하여 호스트를 초기 상태로 롤백합니다. 이를 통해 스크립트의 반복 실행이 가능한 멍등성(Idempotency)을 완벽하게 보장합니다.



※ VNC를 통한 가상머신 접속 확인



4. 자동 테스트 결과 증명

- `04_test.sh` 실행 결과: 요구사항의 통신 격리 규칙을 만족

```

root@kvm-host01:~/workspace# sh 04_test.sh
=== 04. 가상 네트워크 (VNet) L2 통신 격리 테스트 시작 ===
-----
[테스트 1] vm-1 (192.168.0.1) -> vm-2 (192.168.0.2)
기대 결과 : 통신 가능 (동일 vnet-A)
☑결과 : 성공 (통신 가능)
-----
[테스트 2] vm-2 (192.168.0.2) -> vm-3 (192.168.0.3)
기대 결과 : 통신 불가능 (vnet-A와 vnet-B 격리)
☑결과 : 성공 (통신 불가능 - 격리 확인)
-----
[테스트 3] vm-3 (192.168.0.3) -> vm-1 (192.168.0.1)
기대 결과 : 통신 불가능 (vnet-B와 vnet-A 격리)
☑결과 : 성공 (통신 불가능 - 격리 확인)
-----
=== 통신 테스트 완료 ===

```

5. 한계점 및 향후 과제 (Future Works)

본 프로젝트는 제한된 시간과 중첩 가상화(Nested Virtualization) 환경의 특성을 고려하여, 핵심 요구사항인 L2 네트워크 분리와 Ceph RBD 기반의 안정적인 프로비저닝, 그리고 테스트 자동화 구현에 집중했습니다. 향후 실제 베어메탈 서버 환경이 주어진다면 선택 요구사항인 NUMA 자원 배정 및 스토리지 QoS 등을 추가로 고도화할 수 있는 구조로 스크립트를 모듈화 하였습니다.